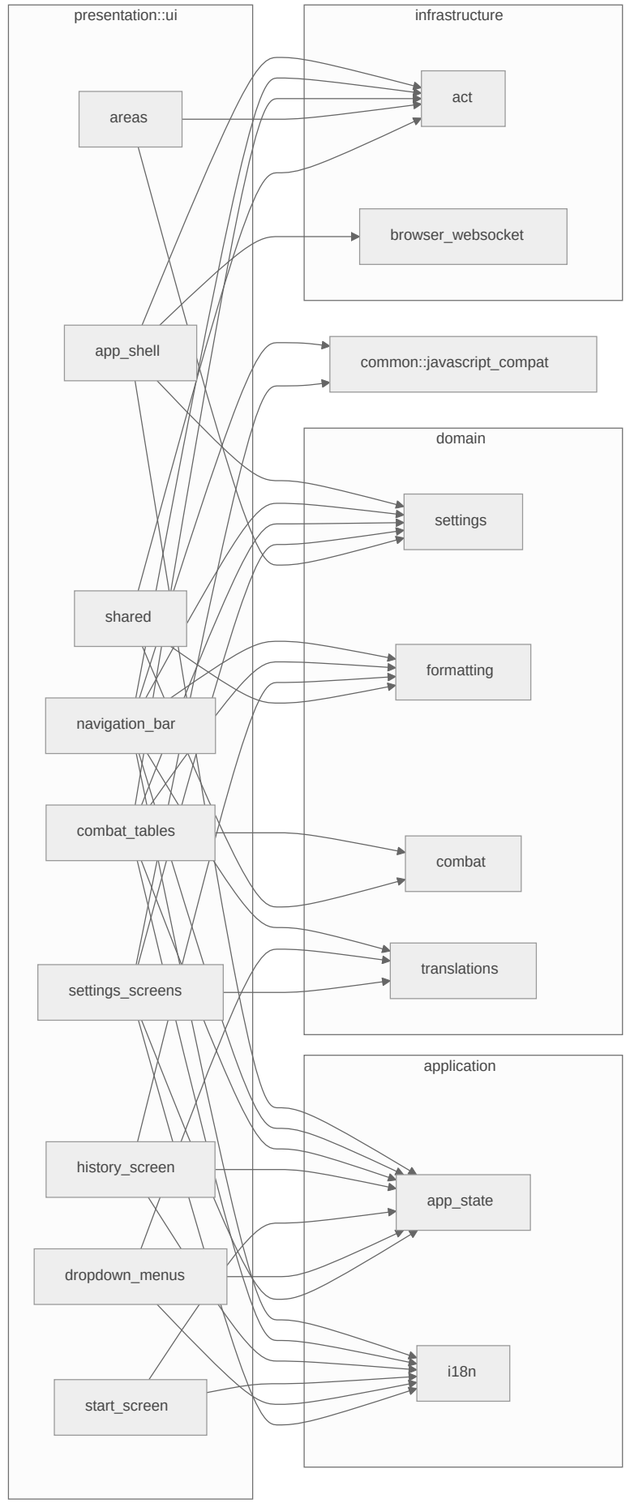
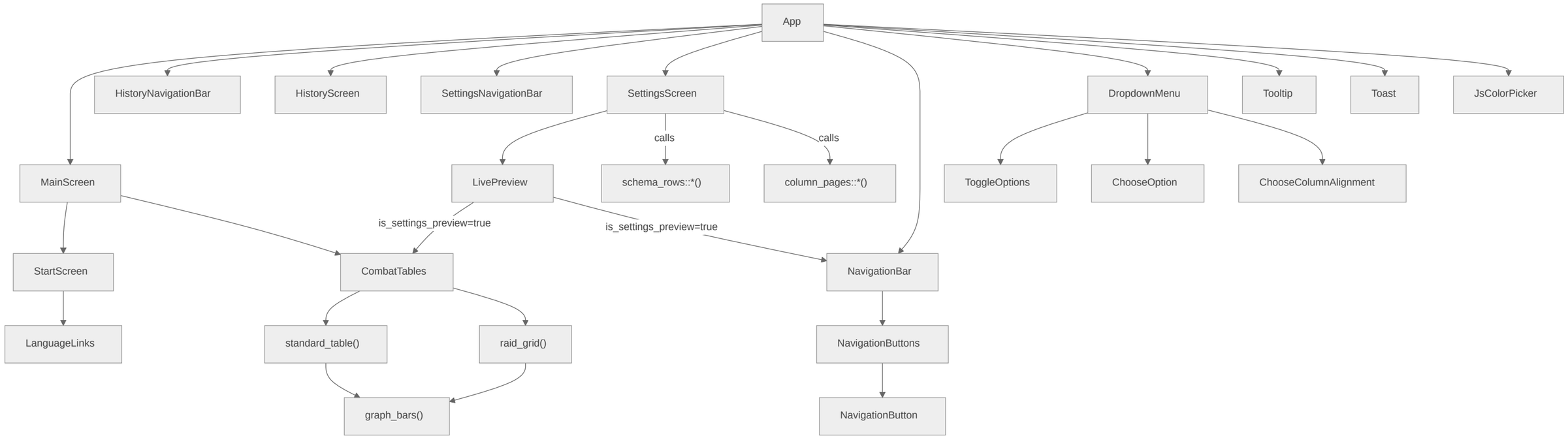
Package dependencies

What each folder under ui/ reaches outside presentation. Arrows point from the user to the used package. Only folders that have an outside dependency are connected.



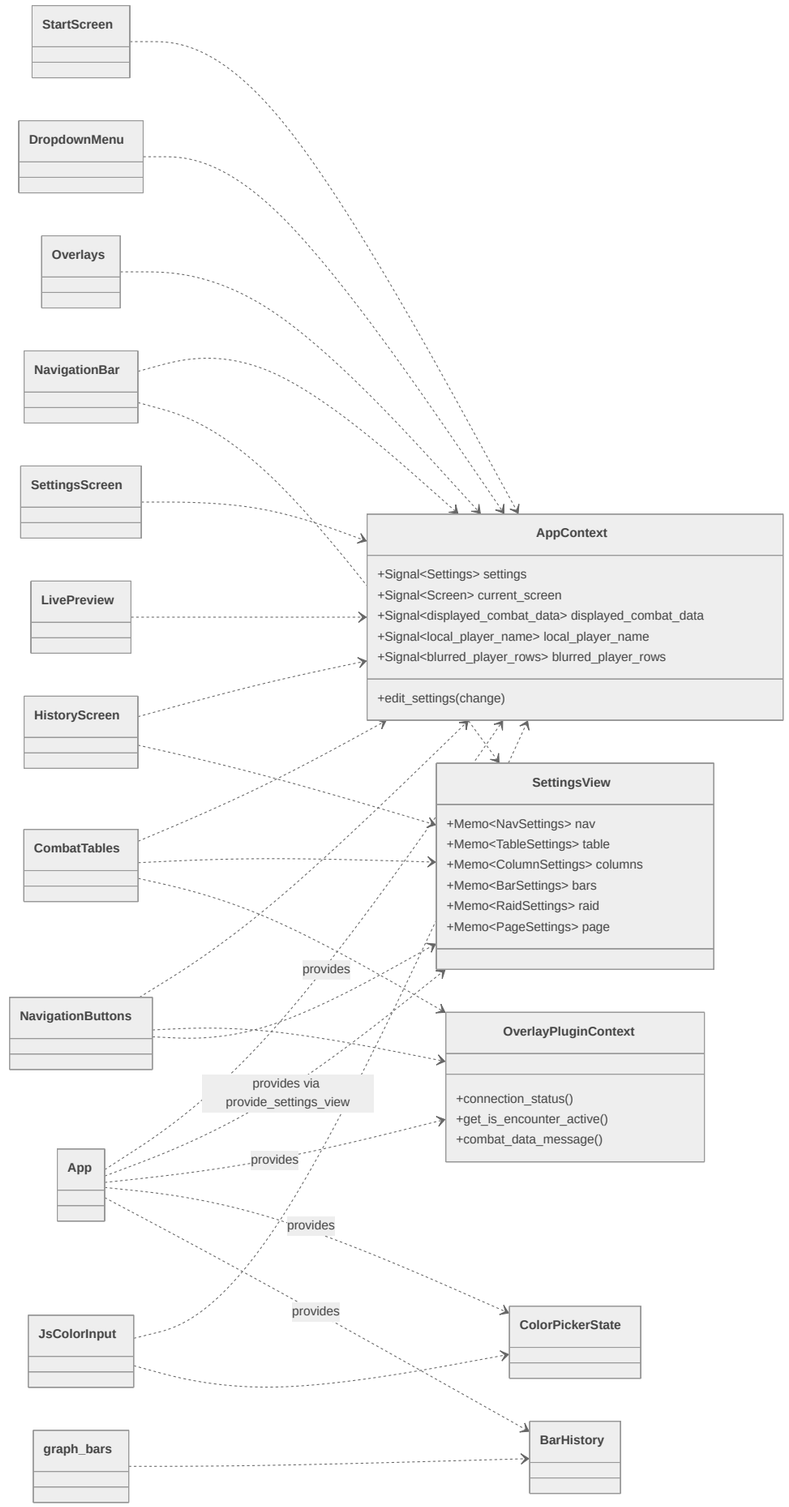
Component render tree

App picks the top bar and the body from AppContext.current_screen (Main, History or Settings). The same two components, CombatTables and NavigationBar, are reused by the main screen and by the settings live preview.



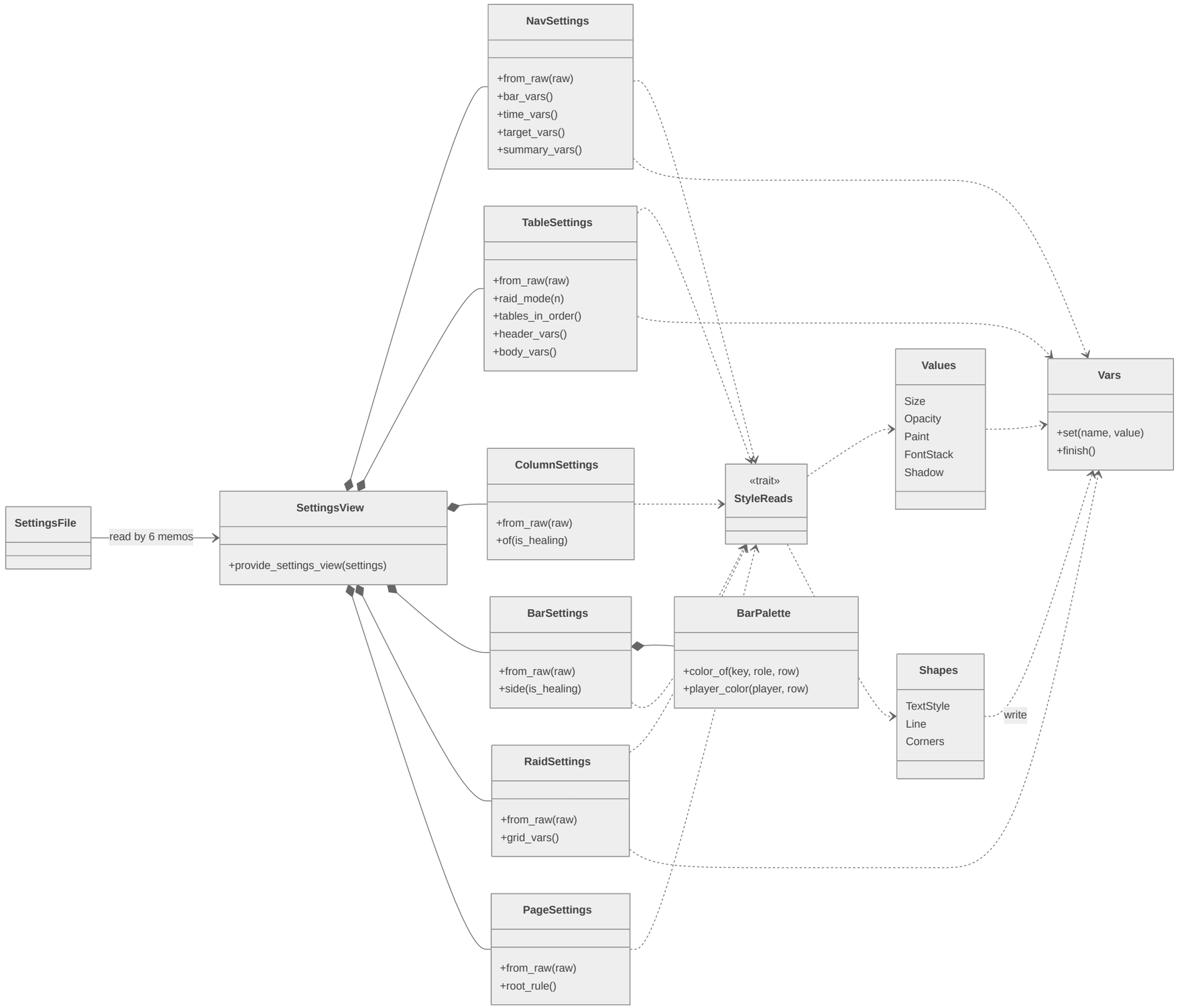
Shared state and who reads it

App creates and provides five values with use_context_provider. Everything below reads them with use_context.



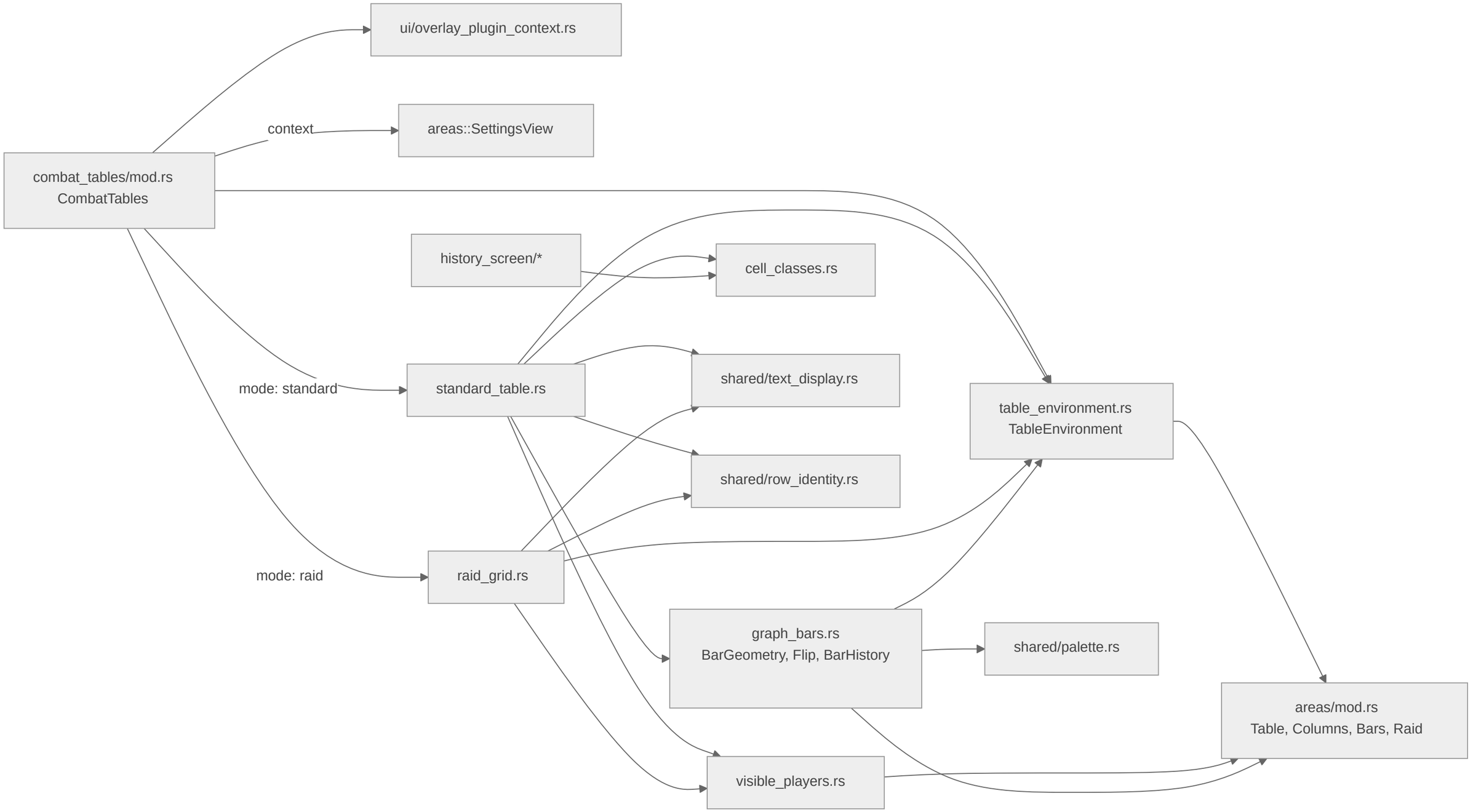
Settings to style pipeline

The refactor's core path: the raw settings file becomes six typed area structs, each of which turns itself into inline CSS variables through the shared style types.



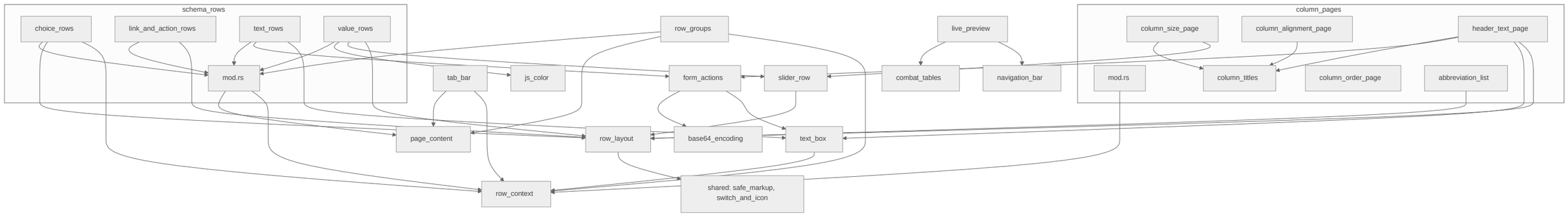
combat_tables, file by file

How CombatTables builds a table. TableEnvironment bundles the area structs and the cell context so the drawing functions take one argument.



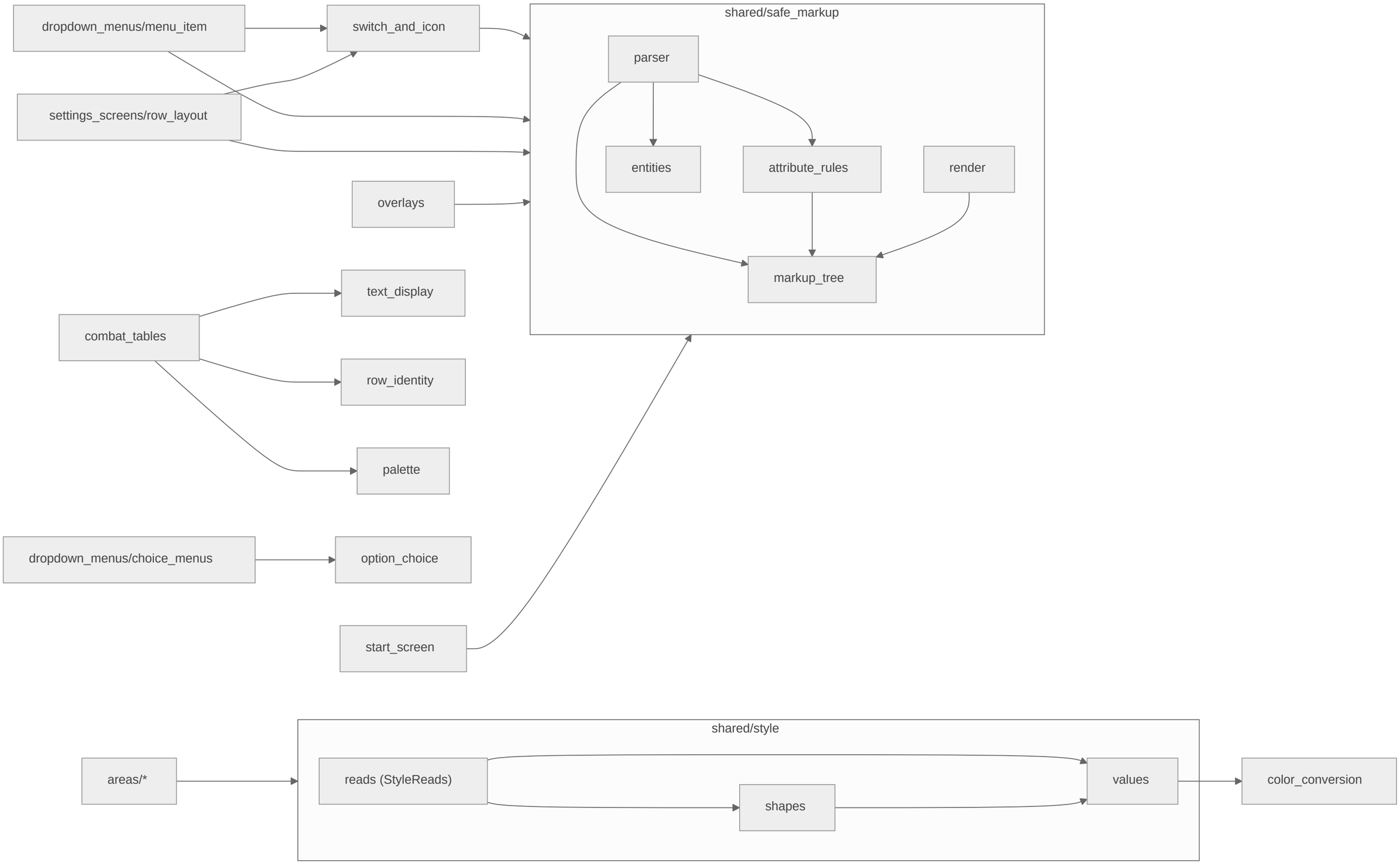
settings_screens, file by file

Pages are driven by data (data/l.json), so schema_rows has one file per row kind. Almost everything depends on row_context and row_layout.



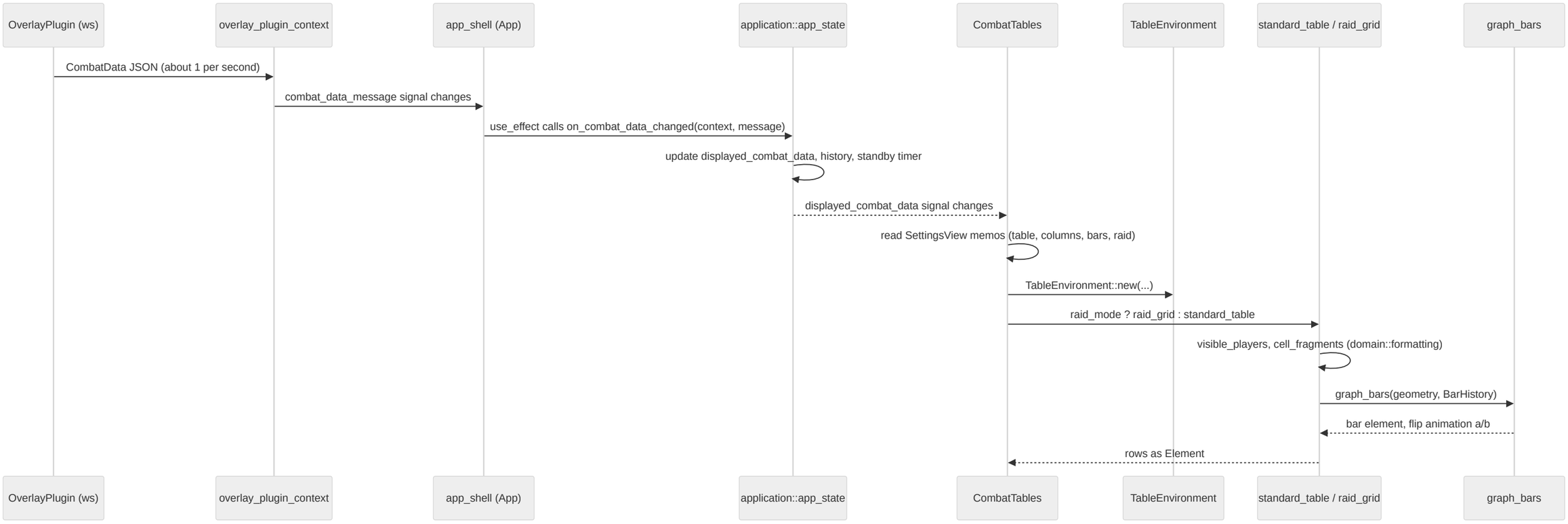
shared

Leaf helpers. style is used only by areas; safe_markup turns the translated HTML snippets into Dioxus elements.



A data update, end to end

ACT sends a message; the table redraws. Only the memos and signals that changed cause work.



Editing a setting

A slider or colour change on a settings page. Because each area is its own memo, only the areas whose values changed re-render their consumers.

